FuelEU Maritime Compliance Platform

Project Report

Varuna Marines Assignment

Submitted by:
Rajneesh Kumar

Department of Physics
IIT Guwahati

April 2026

Contents

1 Project Overview and Assignment Requirements **3**

1.1 Project Details . . . 3

1.2 Overview . . . 3

1.3 Assignment Requirements . . . 4

1.4 Dashboard Modules . . . 4

1.4.1 Routes Module . . . 4

1.4.2 Compare Module . . . 4

1.4.3 Banking Module . . . 4

1.4.4 Pooling Module . . . 5

2 User Interface Screenshots **6**

3 AI Agent Workflow **8**

3.1 Introduction . . . 8

3.2 Tools Used . . . 8

3.3 Approach to Using AI Tools . . . 8

3.4 Examples of Usage . . . 9

3.4.1 Code Generation . . . 9

3.4.2 Refinement and Debugging . . . 9

3.5 Validation and Verification . . . 9

3.6 Observations . . . 10

3.7 Best Practices Followed . . . 10

3.8 Conclusion . . . 10

4 Mathematical Model **11**

4.1 Core Mathematical Formulations . . . 11

4.2 Target GHG Intensity . . . 11

4.3 Energy in Scope . . . 11

4.4 Compliance Balance (CB) . . . 12

4.5 Worked Example . . . 12

4.6 Percent Difference Calculation . . . 13

4.7 Banking Mechanism (Article 20) . . . 13
4.8 Pooling Mechanism (Article 21) . . . 13
4.8.1 Validation Rules . . . 13
4.9 Greedy Allocation Algorithm . . . 14

5 System Architecture **15**
5.1 Overview . . . 15
5.2 Architecture Principles . . . 15
5.3 Architecture Layers . . . 15
5.3.1 Domain Core . . . 16
5.3.2 Ports (Interfaces) . . . 16
5.3.3 Adapters . . . 16
5.4 Backend Architecture . . . 17
5.5 Frontend Architecture . . . 17
5.6 Monorepo Structure . . . 18
5.7 Data Flow . . . 18
5.8 Advantages of the Architecture . . . 18

6 Database Schema **19**
6.1 Overview . . . 19
6.2 Routes Table . . . 19
6.3 Bank Entries Table . . . 20
6.4 Pools Table . . . 20
6.5 Pool Members Table . . . 20

7 API Reference **21**
7.1 Overview . . . 21
7.2 Routes Endpoints . . . 21
7.3 Compliance Endpoints . . . 22
7.4 Banking Endpoints (Article 20) . . . 22
7.5 Pooling Endpoints (Article 21) . . . 23
7.6 Error Handling and Response Format . . . 23

1. Project Overview and Assignment Requirements

1.1 Project Details

Field	Details
Project Title	FuelEU Maritime Compliance Dashboard
Regulation	EU 2023/1805 — FuelEU Maritime
Technology Stack	React, TypeScript, Node.js, PostgreSQL, Drizzle ORM
Architecture	Hexagonal Architecture (Ports and Adapters / Clean Architecture)
Date	April 2026

Table 1.1: Project Information

1.2 Overview

The FuelEU Maritime Compliance Platform is a full-stack web application developed to implement the compliance mechanisms defined in EU Regulation 2023/1805 (FuelEU Maritime). The regulation requires ships operating at European Union ports to meet progressively stricter greenhouse gas (GHG) intensity targets over time.

This platform provides a centralized dashboard for compliance officers to monitor vessel routes, calculate compliance balances, and apply regulatory flexibility mechanisms such as banking and pooling. The system is designed to simplify annual compliance management and ensure adherence to regulatory requirements through accurate calculations and clear data visualization.

Component	Specification	Status
Frontend	React + TypeScript + TailwindCSS	Implemented
Backend	Node.js + TypeScript + PostgreSQL	Implemented
Architecture	Hexagonal (Ports and Adapters)	Implemented
Routes Tab	Table, Baseline selection, Filters	Implemented
Compare Tab	Baseline comparison, charts, percentage difference	Implemented
Banking Module	Article 20 (Bank / Apply surplus)	Implemented
Pooling Module	Article 21 (Pool creation and validation)	Implemented
Compliance Formula	Annex IV calculation	Implemented
Database	Five tables with seeded data	Implemented
API Integration	OpenAPI with Zod and React Query	Implemented
Documentation	README and supporting documents	Delivered

Table 1.2: Assignment Requirements and Implementation Status

1.3 Assignment Requirements

1.4 Dashboard Modules

1.4.1 Routes Module

The Routes module displays all predefined routes from the dataset. It provides filtering options based on vessel type, fuel type, and year. Each entry includes a *Set Baseline* option, which allows the user to define a reference route. Selecting a new baseline automatically resets any previously selected baseline.

1.4.2 Compare Module

The Compare module enables comparison between the baseline route and other routes. It visualizes GHG intensity using a bar chart, along with reference lines for the baseline and the regulatory target value of 89.3368 gCO_2e/MJ. A tabular view presents the percentage deviation from the baseline and indicates whether each route meets compliance requirements.

1.4.3 Banking Module

The Banking module implements the flexibility mechanism described in Article 20 of the regulation. Users can view the compliance balance for a selected ship and year, and perform actions such as banking surplus or applying previously banked values. These actions are enabled only when valid conditions are met. A transaction history is also maintained for transparency.

1.4.4 Pooling Module

The Pooling module supports the mechanism defined in Article 21. Multiple ships can be grouped into a pool to collectively meet compliance targets. The system validates that the total adjusted compliance balance is non-negative, ensures that deficit ships are not adversely affected, and prevents surplus ships from becoming non-compliant. A greedy allocation algorithm is used to distribute surplus among deficit ships.

2. User Interface Screenshots

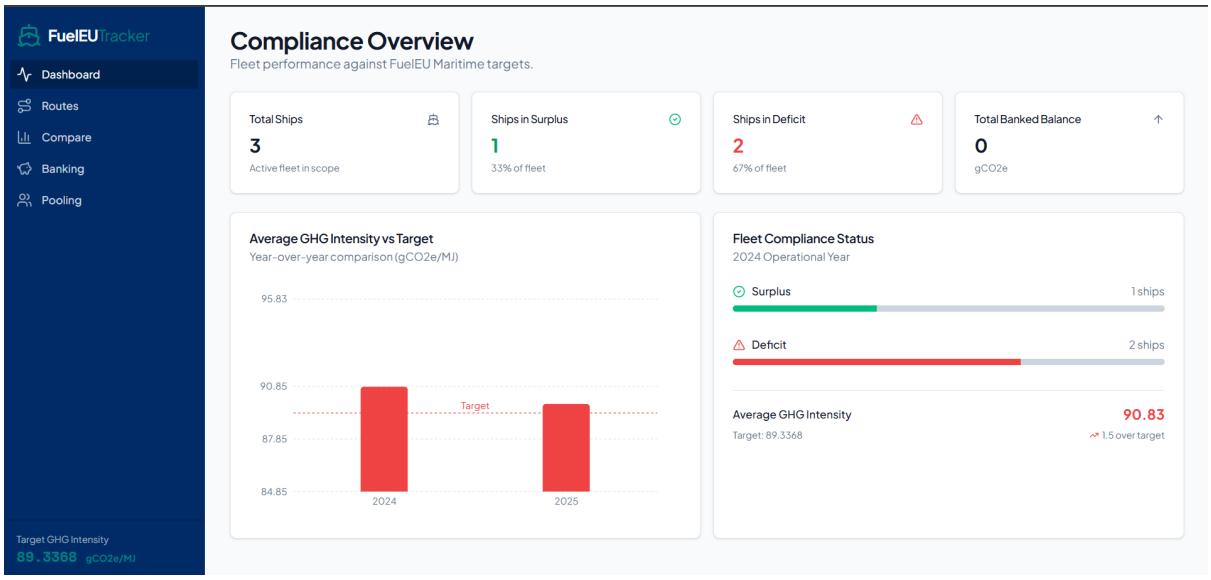

Figure 2.1: Dashboard View

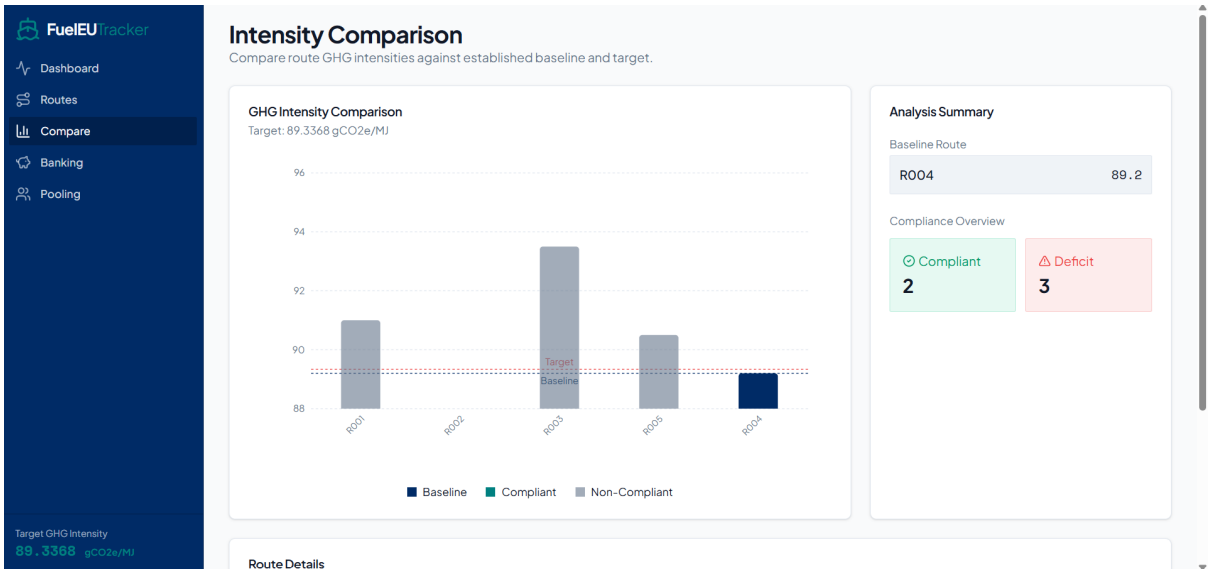

Figure 2.2: Compare Page

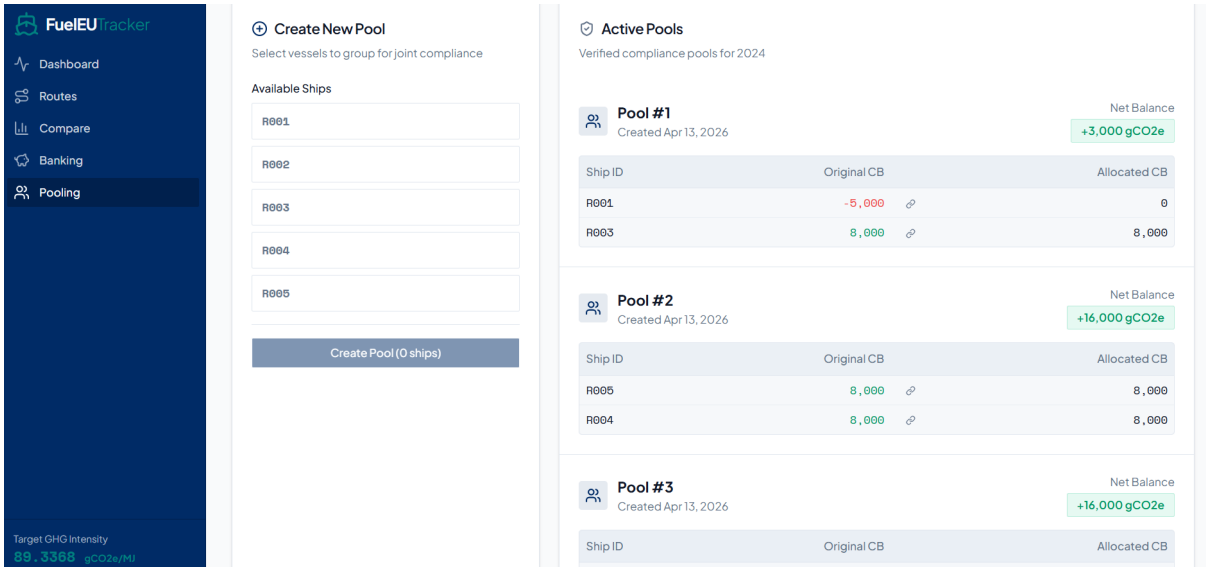

Figure 2.3: Pooling Page

3. AI Agent Workflow

3.1 Introduction

During the development of this project, AI-assisted tools were used as supporting instruments to improve productivity, explore alternative implementations, and accelerate routine development tasks. The goal was not to rely blindly on generated code, but to use these tools as assistants while maintaining full control over the design, logic, and correctness of the system.

3.2 Tools Used

The following AI tools were used at different stages of development:

- **GitHub Copilot** — used for inline code suggestions, especially for boilerplate code, API handlers, and repetitive structures.
- **ChatGPT** — used for understanding architectural patterns, refining logic, and resolving complex issues related to compliance calculations and system design.
- **Cursor IDE** — used for structured code generation and refactoring larger blocks of code, particularly in organizing modules according to hexagonal architecture.

Each tool was used selectively depending on the task, rather than relying on a single source for all development.

3.3 Approach to Using AI Tools

AI tools were integrated into the workflow in a controlled and iterative manner. The typical process followed was:

1. Define the problem clearly (e.g., compliance balance calculation, pooling validation rules)
2. Use AI tools to generate an initial implementation or suggestion

3. Manually review the generated code for correctness and alignment with requirements
4. Refactor and simplify the solution where necessary
5. Test the logic with different scenarios to ensure correctness

This approach ensured that the final implementation remained consistent with the project requirements and domain logic.

3.4 Examples of Usage

3.4.1 Code Generation

AI tools were used to generate initial versions of:

- API route handlers
- Database schema definitions
- TypeScript interfaces and types

These outputs significantly reduced the time required to write repetitive code, especially in setting up the backend structure.

3.4.2 Refinement and Debugging

In several cases, AI-generated code required refinement. For example:

- Adjusting compliance balance calculations to match exact formula definitions
- Fixing edge cases in banking logic (e.g., preventing over-application of banked surplus)
- Correcting pooling logic to ensure all validation rules were satisfied

AI suggestions were treated as drafts rather than final solutions.

3.5 Validation and Verification

All generated code was carefully verified through:

- Manual inspection of logic and formulas

- Testing with sample data (including edge cases such as negative compliance balance)
- Cross-checking results with expected outputs from the specification

This step was essential to ensure that the implementation strictly followed the FuelEU Maritime regulation.

3.6 Observations

The use of AI tools provided several benefits:

- Faster development of repetitive and boilerplate code
- Improved understanding of architectural patterns
- Assistance in structuring complex logic into manageable components

However, some limitations were also observed:

- AI occasionally produced incorrect or incomplete logic for domain-specific rules
- Generated code sometimes required simplification or restructuring
- Careful validation was necessary to avoid subtle errors

3.7 Best Practices Followed

To ensure effective use of AI tools, the following practices were followed:

- Avoided copying code without understanding its logic
- Broke complex problems into smaller, well-defined tasks
- Used AI primarily as an assistant, not as a decision-maker
- Verified all critical computations independently

3.8 Conclusion

AI-assisted development significantly improved efficiency in this project, particularly in reducing development time and assisting with code structure. However, the final responsibility for correctness, design decisions, and implementation quality remained with the developer. The combination of human reasoning and AI assistance resulted in a balanced and effective development process.

4. Mathematical Model

4.1 Core Mathematical Formulations

The mathematical formulations used in this project are derived from Annex IV of EU Regulation 2023/1805 and the ESSF guidance document. All formulas are implemented in a standalone TypeScript module (`compliance.ts`), which serves as the domain core of the system. This module is independent of any framework, ensuring modularity and testability.

4.2 Target GHG Intensity

The baseline greenhouse gas (GHG) intensity for the reference year 2020 is defined as:

$$91.16 \text{ gCO}_2\text{e/MJ} \tag{4.1}$$

The target GHG intensity for the year 2025 is set to be 2% below the baseline:

$$\text{Target Intensity} = 91.16 \times (1 - 0.02) = 89.3368 \text{ gCO}_2\text{e/MJ} \tag{4.2}$$

This value is used as a constant in the implementation.

4.3 Energy in Scope

The total energy in scope is calculated based on fuel consumption using the Lower Heating Value (LHV) approximation:

$$E_{\text{scope}} = \text{Fuel Consumption (tonnes)} \times 41{,}000 \text{ MJ/t} \tag{4.3}$$

This represents the total energy used and serves as the denominator for GHG intensity calculations.

4.4 Compliance Balance (CB)

The Compliance Balance (CB) is the primary metric used to evaluate whether a ship meets the regulatory target. It is defined as:

$$\text{CB} = (\text{Target Intensity} - \text{Actual Intensity}) \times E_{\text{scope}} \tag{4.4}$$

Expanding the formula:

$$\text{CB} = (89.3368 - \text{GHG}_{\text{actual}}) \times (\text{Fuel Consumption} \times 41{,}000) \tag{4.5}$$

Interpretation:

- CB > 0: Surplus (compliant)
- CB < 0: Deficit (non-compliant)
- CB $= 0$: Exactly compliant

4.5 Worked Example

Consider Route R002 (Bulk Carrier, LNG, Year 2024):

- Actual GHG Intensity = 88.0 gCO_2e/MJ
- Fuel Consumption = 4800 tonnes

Step 1: Energy in Scope

$$E_{\text{scope}} = 4800 \times 41{,}000 = 196{,}800{,}000 \text{ MJ} \tag{4.6}$$

Step 2: Compliance Balance

$$\text{CB} = (89.3368 - 88.0) \times 196{,}800{,}000 \tag{4.7}$$

$$\text{CB} = 263{,}082{,}240 \text{ gCO}_2\text{e} \tag{4.8}$$

Result: The route has a positive compliance balance, indicating a surplus and compliance with the regulation.

4.6 Percent Difference Calculation

The percentage difference used in route comparison is defined as:

$$\text{Percent Difference} = \left(\frac{\text{GHG}_{\text{comparison}}}{\text{GHG}_{\text{baseline}}} - 1 \right) \times 100 \tag{4.9}$$

Interpretation:

- Positive value: Worse than baseline
- Negative value: Better than baseline

4.7 Banking Mechanism (Article 20)

The banking mechanism allows ships with surplus compliance balance to store it for future use.

Key Concept:

$$\text{Available Banked} = \text{Total Banked} - \text{Total Applied} \tag{4.10}$$

Constraints:

- Banking is allowed only if CB > 0
- Applied amount must not exceed available banked balance

4.8 Pooling Mechanism (Article 21)

Pooling allows multiple ships to combine their compliance balances.

4.8.1 Validation Rules

A pool is considered valid only if all the following conditions are satisfied:

1. Sum of adjusted compliance balances ≥ 0
2. Deficit ships must not exit with worse balance:

$$\text{CB}_{after} \geq \text{CB}_{before} \tag{4.11}$$

3. Surplus ships must not become negative:

$$\text{CB}_{after} \geq 0 \tag{4.12}$$

4.9 Greedy Allocation Algorithm

When a valid pool is formed, surplus compliance balance is distributed to deficit ships using a greedy allocation strategy.

Algorithm Steps:

- Sort ships in descending order of compliance balance
- Calculate total available surplus
- Iteratively allocate surplus to deficit ships
- Ensure no surplus ship becomes negative

This approach prioritizes efficient distribution of surplus while maintaining all regulatory constraints.

5. System Architecture

5.1 Overview

The system is designed using **Hexagonal Architecture**, also known as *Ports and Adapters* or *Clean Architecture*. This approach ensures that the core business logic is completely independent of external frameworks such as web servers, databases, and user interfaces.

The architecture promotes modularity, testability, and maintainability by clearly separating the domain logic from infrastructure concerns.

5.2 Architecture Principles

The following key principles guide the system design:

- **Separation of Concerns:** Business logic is isolated from UI, database, and API layers.
- **Dependency Inversion:** Core logic depends only on abstract interfaces (ports), not on implementations.
- **Testability:** Domain logic can be tested independently without requiring a running server or database.
- **Scalability:** Components can be extended or replaced without affecting the core system.

5.3 Architecture Layers

The system is divided into three main layers:

5.3.1 Domain Core

The domain core contains the fundamental business logic and is completely independent of any framework.

- Implements all compliance-related calculations such as:
  - Compliance Balance (CB)
  - Energy in Scope
  - Percent Difference
  - Pool Allocation Logic
- Located in: `compliance.ts`
- Contains pure TypeScript functions with no external dependencies

5.3.2 Ports (Interfaces)

Ports define the communication boundaries of the system. They act as contracts between the domain core and external systems.

- API contracts defined using OpenAPI specification
- Input/output validation using Zod schemas
- Ensure consistent communication between frontend and backend

5.3.3 Adapters

Adapters implement the ports and connect the system to external components.

- **Inbound Adapters:**
  - HTTP routes using Express
  - Handle incoming API requests
- **Outbound Adapters:**
  - Database interaction using PostgreSQL and Drizzle ORM
  - External data persistence
- **Frontend Adapter:**
  - React-based user interface
  - Uses generated API hooks for communication

5.4 Backend Architecture

The backend follows a layered structure aligned with hexagonal principles:

- **Domain Layer:** Core business logic implemented in `compliance.ts`
- **API Layer:** OpenAPI specification acts as a single source of truth
- **HTTP Layer:** Express routes handle incoming requests and responses
- **Database Layer:** Drizzle ORM manages PostgreSQL interactions

Each request follows a structured flow:

1. Request is received via an HTTP route
2. Input is validated using Zod schemas
3. Business logic is executed in the domain core
4. Data is fetched or stored via database adapters
5. Response is returned to the client

5.5 Frontend Architecture

The frontend is implemented using React and follows a modular structure:

- **Pages:** Represent different dashboard views (Routes, Compare, Banking, Pooling)
- **Components:** Reusable UI elements
- **API Layer:** Generated React Query hooks for data fetching
- **State Management:** Managed using React Query for server state

The frontend does not contain business logic; it only interacts with the backend via API calls and displays processed results.

5.6 Monorepo Structure

The project is organized as a monorepo using pnpm workspaces:

```
/
 artifacts/
    fueleu-dashboard/   (React frontend)
    api-server/         (Express backend)
 lib/
    api-spec/           (OpenAPI specification)
    api-client-react/   (Generated API hooks)
    api-zod/            (Validation schemas)
    db/                 (Database schema and client)
```

5.7 Data Flow

The data flow in the system follows a structured pipeline:

1. User interacts with the frontend interface
2. Frontend sends request using API hooks
3. Backend validates request parameters
4. Domain core processes business logic
5. Database adapter handles data retrieval/storage
6. Response is returned to frontend
7. UI updates dynamically with new data

5.8 Advantages of the Architecture

- Clear separation between logic and infrastructure
- High maintainability and scalability
- Easy integration of new features
- Improved testability of core logic
- Reduced dependency on specific frameworks

6. Database Schema

6.1 Overview

The database schema is implemented using PostgreSQL and managed through Drizzle ORM. The schema is defined programmatically using TypeScript, allowing strong type safety and consistency across the application.

Drizzle-Zod is used to automatically generate validation schemas from the database definitions, eliminating redundancy and ensuring synchronization between database and API layers.

6.2 Routes Table

The `routes` table stores voyage-related data and serves as the core entity for compliance calculations.

Column	**Type**	**Constraint**	**Description**
id	serial	Primary Key	Auto-incrementing identifier
route_id	text	UNIQUE, NOT NULL	Business identifier (R001–R005)
vessel_type	text	NOT NULL	Vessel category (Container, BulkCarrier, etc.)
fuel_type	text	NOT NULL	Fuel type (HFO, LNG, MGO)
year	integer	NOT NULL	Compliance year
ghg_intensity	real	NOT NULL	GHG intensity (gCO_2e/MJ)
fuel_consumption	real	NOT NULL	Fuel consumed (tonnes)
distance	real	NOT NULL	Route distance (km)
total_emissions	real	NOT NULL	Total emissions (tonnes CO_2e)
is_baseline	boolean	NOT NULL	Indicates baseline route
ship_id	text	NOT NULL	Ship identifier
energy_in_scope	real	NOT NULL	Computed energy (MJ)

Table 6.1: Routes Table Schema

6.3 Bank Entries Table

The `bank_entries` table stores all banking transactions, including surplus storage and application.

Column	Type	Constraint	Description
id	serial	Primary Key	Auto-incrementing identifier
ship_id	text	NOT NULL	Ship performing the transaction
year	integer	NOT NULL	Compliance year
amount_gco2eq	real	NOT NULL	Amount (gCO_2e)
entry_type	text	NOT NULL	'banked' or 'applied'
created_at	timestamptz	NOT NULL	Timestamp of transaction

Table 6.2: Bank Entries Table Schema

6.4 Pools Table

The `pools` table maintains records of all compliance pools created.

Column	Type	Constraint	Description
id	serial	Primary Key	Auto-incrementing identifier
year	integer	NOT NULL	Compliance year
created_at	timestamptz	NOT NULL	Pool creation timestamp

Table 6.3: Pools Table Schema

6.5 Pool Members Table

The `pool_members` table stores the details of each ship participating in a pool, including compliance balance before and after allocation.

Column	Type	Constraint	Description
id	serial	Primary Key	Auto-incrementing identifier
pool_id	integer	NOT NULL	Foreign key referencing pools.id
ship_id	text	NOT NULL	Participating ship
cb_before	real	NOT NULL	CB before allocation (gCO_2e)
cb_after	real	NOT NULL	CB after allocation (gCO_2e)

Table 6.4: Pool Members Table Schema

7. API Reference

7.1 Overview

All API endpoints are exposed under the base URL `/api`. The API follows an OpenAPI 3.1 specification defined in `lib/api-spec/openapi.yaml`. This contract-first approach ensures consistency between the frontend and backend and enables automatic generation of validation schemas and client-side hooks.

7.2 Routes Endpoints

Method	Path	Parameters	Description
GET	/routes	vesselType?, fuelType?, year?	Returns all routes with computed energy in scope and compliance balance. Supports filtering.
GET	/routes/comparison	–	Returns baseline route along with all other routes, including percentage difference and compliance status.
POST	/routes/:routeId/baseline	routeId (path)	Sets the selected route as baseline and clears any previously defined baseline.

Table 7.1: Routes API Endpoints

7.3 Compliance Endpoints

Method	Path	Parameters	Description
GET	/compliance/cb	shipId, year	Computes and returns the compliance balance (CB) along with target intensity, actual intensity, energy in scope, and status.
GET	/compliance/adjusted-cb	shipId, year	Returns compliance balance after applying banking transactions, including total banked and applied values.
GET	/compliance/summary	year	Provides fleet-level summary including total ships, surplus/deficit counts, average GHG intensity, and target intensity.

Table 7.2: Compliance API Endpoints

7.4 Banking Endpoints (Article 20)

Method	Path	Parameters	Description
GET	/banking/records	shipId, year	Retrieves the complete banking transaction history for a given ship and year.
POST	/banking/bank	shipId, year, amount	Stores surplus compliance balance. Validates that the amount does not exceed available surplus.
POST	/banking/apply	shipId, year, amount	Applies previously banked surplus to offset a deficit. Ensures sufficient available balance.

Table 7.3: Banking API Endpoints

7.5 Pooling Endpoints (Article 21)

Method	Path	Parameters	Description
POST	/pools	year, members	Creates a compliance pool. Validates pool constraints and applies greedy allocation to distribute surplus among deficit ships.
GET	/pools	year (optional)	Returns all pools, optionally filtered by year, including compliance balances before and after allocation.

Table 7.4: Pooling API Endpoints

7.6 Error Handling and Response Format

- All responses are returned in JSON format.
- Input and output validation is performed using Zod schemas generated from the OpenAPI specification.
- Error responses follow a consistent structure:

```
{ "error": "description" }
```

- Common HTTP status codes:
  - 200 — Success
  - 400 — Bad Request (validation failure)
  - 404 — Resource not found
  - 500 — Internal server error